

Auditoría de Seguridad Cloud

Aplicación ArchivaCloud (S3 File Upload)

Frontend React + Backend Python/FastAPI · Amazon S3

Integrante: Joaquín Eduardo Zambrano Baeza

Grupo: Tema1 (**Fundamentos web y comunicación segura**)

Asignatura: Arquitectura On-Premise y On-Cloud

Docente: Yasna León Foitzick

Fecha de entrega: 02 / 07 / 2026

Repositorio del proyecto: <https://github.com/JoaquinEZB-7/auditoria-archivacloud-p10.git>

Tabla de contenido

Tabla de contenido	2
Índice de figuras y tablas	4
Glosario de términos.....	4
1. Resumen ejecutivo.....	6
2. Introducción.....	7
2.1 Contexto del encargo.....	7
2.2 Objetivos	7
2.3 Alcance y consideraciones éticas	7
2.4 Marco metodológico.....	7
2.5 Arquitectura de la aplicación auditada.....	8
3. Fase 1 — Análisis estático (SAST/SCA)	9
3.1 Metodología y herramientas	9
3.2 Revisión manual del código	9
3.3 Análisis automatizado e interpretación	11
3.4 Hallazgos de dependencias (SCA)	11
3.5 Discrepancia entre la documentación de seguridad y la implementación real.....	11
3.6 Tabla resumen de hallazgos.....	12
4. Fase 2 — Modelado de amenazas	13
4.1 Diagrama de flujo de datos y fronteras de confianza	13
4.2 Matriz STRIDE.....	14
4.3 Priorización (CVSS v3.1)	14
4.4 Mapeo MITRE ATT&CK for Cloud.....	15
4.5 Caso de abuso encadenado	15
5. Fase 3 — Pruebas dinámicas (DAST / pentesting controlado).....	16
5.1 Entorno y reglas de ejecución.....	16
5.2 Resultados por vector	16
5.3 Síntesis de pruebas	18
6. Fase 4 — Auditoría del despliegue en la nube.....	19
6.1 IAM y credenciales	19
6.2 Configuración del bucket S3	19
6.3 CORS del bucket.....	19
6.4 Auditoría contra CIS AWS Foundations Benchmark v3.0.....	20
6.5 Evaluación contra el AWS Well-Architected Framework.....	20
7. Fase 5 — Plan de operación y mantención segura (NIST CSF 2.0).....	21
7.1 Gobernar	21
7.2 Identificar	21

7.3 Proteger	21
7.4 Detectar	22
7.5 Responder	22
7.6 Recuperar	22
7.7 Matriz RACI y cronograma	22
8. Fase 6 — Propuesta de arquitectura segura.....	23
8.1 Diagrama de arquitectura objetivo.....	23
8.2 Decisiones y justificación	24
8.3 Infraestructura como código (IaC)	24
8.4 Estimación de costos AWS	25
8.5 Trade-offs.....	26
9. Conclusiones	26
10. Catálogo de vulnerabilidades.....	27
11. Bibliografía (APA 7)	34
Anexos.....	35
Anexo A — Evidencias.....	35
Anexo B — Matriz STRIDE completa.....	46
Anexo C — Código IaC.....	46
Anexo D — Declaración de uso de IA generativa	46

Índice de figuras y tablas

Evidencia 1 Bandit: sin hallazgos (Fase 1).....	35
Evidencia 2 Código: key=f"uploads/{fileName}" (VULN-002).....	36
Evidencia 3 Código: endpoint DELETE (VULN-001/005).....	36
Evidencia 4 presigned-url + str(e) (VULN-002/004/005).....	37
Evidencia 5 Código: list_files (VULN-001/005).....	38
Evidencia 6 pip-audit: 8 vuln. en 3 paquetes (VULN-006/007).....	38
Evidencia 7 Auditoría bucket: cifrado, versioning, BPA, logging, lifecycle, policy (Fase 4).....	39
Evidencia 8 Auditoría: IAM (voclabs) y CORS del bucket (VULN-012).....	40
Evidencia 9 npm audit: 0 vulnerabilidades (Fase 1).....	40
Evidencia 10 Semgrep: 0 hallazgos (Fase 1).....	41
Evidencia 11 DELETE sobre inexistente → "Archivo eliminado" (observación).....	41
Evidencia 12 GET /api/files sin token (VULN-001).....	41
Evidencia 13 Path traversal: key con ../ (VULN-002).....	42
Evidencia 14 DELETE sin token (VULN-001).....	42
Evidencia 15 html aceptado como PDF (VULN-004).....	43
Evidencia 16 Archivo malicioso almacenado en la app (VULN-004).....	44
Evidencia 17 Error crudo de boto3 al cliente (VULN-005).....	44
Evidencia 18 Sobrescritura confirmada con aws s3 cp (VULN-003).....	45
Evidencia 19 CORS: origen externo bloqueado (control correcto).....	46

Glosario de términos

Término	Significado
Endpoint	Punto de acceso de la aplicación: una dirección a la que se envían peticiones (p. ej. /api/files).
Bucket	Contenedor de almacenamiento de Amazon S3 donde se guardan los archivos.
Presigned URL (URL prefirmada)	Enlace temporal y autorizado que permite subir un archivo directamente a S3 sin pasar por el servidor.
Path traversal (recorrido de rutas)	Técnica de ataque que usa secuencias como ../ para salir de la carpeta permitida.
XSS (Cross-Site Scripting)	Inyección de código malicioso que se ejecuta en el navegador de otro usuario.
Versionado (versioning)	Función que conserva versiones anteriores de un archivo para poder recuperarlas.
Registro de accesos (logging)	Bitácora automática de quién accede o modifica los recursos.
Cifrado en reposo	Protección de los datos almacenados mediante cifrado (SSE-S3 usa clave de AWS; SSE-KMS/CMK, clave propia).
OWASP	Organización que publica las listas de riesgos web (Top 10) y de APIs más conocidas.
SAST / SCA / DAST	Análisis del código / de las dependencias / de la aplicación en ejecución.
STRIDE	Modelo para clasificar amenazas: Suplantación, Manipulación, Repudio, Divulgación, Denegación de servicio y Elevación de privilegios.
CVSS / CWE / CVE	Puntaje de gravedad / catálogo de tipos de debilidad / identificador de una vulnerabilidad conocida en un producto.

Término	Significado
IAM	Servicio de AWS para gestionar identidades y permisos de acceso.
S3 / KMS / CMK	Almacenamiento de archivos / servicio de claves de cifrado / clave gestionada por el cliente (AWS).
CORS / TLS	Reglas de acceso entre sitios web distintos / cifrado de las comunicaciones (HTTPS).
MITRE ATT&CK	Catálogo de técnicas reales usadas por atacantes.
NIST CSF / ISO 27001 / CIS	Marcos y guías de buenas prácticas y controles de ciberseguridad.
RACI / RPO / RTO	Matriz de responsabilidades / cuánta información se puede perder / cuánto puede tardar la recuperación.
IaC (Infraestructura como Código)	Definir la infraestructura de la nube mediante archivos de texto (p. ej. Terraform).
boto3 / SDK	Biblioteca de Python para usar servicios de AWS / kit de herramientas de desarrollo.

1. Resumen ejecutivo

Somos un equipo de consultores externos de ciberseguridad y fuimos contratados por CloudConta SpA para realizar una auditoría de seguridad sobre su más reciente aplicación, ArchivaCloud P-10, desarrollada por su equipo de TI para la gestión de documentos en la nube. El objetivo de esta auditoría fue identificar los riesgos de seguridad del sistema antes de que entre en operación con usuarios y datos reales. En este documento se resumen los resultados en un lenguaje orientado a la gerencia; el detalle técnico se encuentra en el informe completo.

Estado general.

Durante la auditoría se identificaron 13 hallazgos de seguridad validados, de los cuales 1 es de gravedad crítica, 4 de gravedad alta, 7 de gravedad media y 1 de carácter informativo. Debido a esto, se concluye que la aplicación presenta un nivel de seguridad deficiente y que no debería desplegarse en producción sin antes aplicar las correcciones recomendadas.

El riesgo más grave.

El hallazgo más grave es que la aplicación no exige ninguna identificación para operar, por lo que cualquier persona que conozca la dirección del servidor puede consultar, descargar y eliminar todos los documentos almacenados sin necesidad de una cuenta ni contraseña. Esto implica que, desde el momento en que la aplicación se publique, la información de los clientes de CloudConta SpA quedaría completamente expuesta.

Otros riesgos relevantes.

Además, se detectó que un atacante podría reemplazar documentos existentes por versiones falsificadas, y que dicha acción no dejaría rastro ni podría revertirse, ya que la aplicación no conserva versiones anteriores de los archivos ni mantiene un registro de quién accede o modifica la información. También se identificaron componentes de software con fallas de seguridad conocidas que requieren actualización.

Aspectos correctamente implementados.

No todo presenta deficiencias. Se dejó constancia de que el almacenamiento no es accesible públicamente desde internet, de que la aplicación mitiga por diseño un tipo común de ataque en el navegador, y de que las restricciones de acceso entre distintos sitios web están correctamente configuradas. Estos controles constituyen una base sobre la cual construir las mejoras.

Recomendaciones prioritarias.

En orden de impacto, se recomienda lo siguiente:

- Control de acceso: implementar un sistema de autenticación que exija identificación antes de acceder a la aplicación, como medida indispensable previa a cualquier publicación.
- Recuperación de datos: habilitar el historial de versiones del almacenamiento, para poder recuperar los documentos ante una sobreescritura o un borrado.
- Gestión de credenciales: trasladar las credenciales de acceso a la nube a un sistema de gestión segura de secretos, otorgando únicamente los permisos estrictamente necesarios.
- Trazabilidad: activar el registro de accesos, para tener constancia de las acciones realizadas y poder investigar cualquier incidente.
- Mantención: actualizar los componentes de software que presentan vulnerabilidades conocidas.

Conclusión.

Los resultados confirman que ArchivaCloud P-10 requiere una intervención de seguridad importante antes de entrar en producción. Sin embargo, todos los hallazgos identificados son corregibles

mediante controles conocidos y de bajo costo relativo, por lo que la aplicación de estas recomendaciones permitiría llevar el sistema desde su estado vulnerable actual hacia uno acorde a las buenas prácticas de la industria, protegiendo adecuadamente la información de los clientes de CloudConta SpA.

2. Introducción

2.1 Contexto del encargo.

Somos un equipo que opera como consultores externos especializada en el rubro de la Ciberseguridad y fuimos contratados para realizar una auditoria para la empresa CloudConta SpA en su más reciente aplicación desarrollada por su equipo de Ti la cual se llama ArchivaCloud P-10 que permite a usuarios autenticados subir, listar y eliminar archivos listados en un bucket S3

2.2 Objetivos

Objetivo general:

Auditar integralmente la aplicación ArchivaCloud, identificando vulnerabilidades en su implementación, despliegue y operación, y proponer un plan de remediación basado en marcos y estándares de la industria.

Objetivos específicos:

- Analizar estáticamente el código del frontend y backend, mapeando cada hallazgo a OWASP Top 10 (2021) y API Security Top 10 (2023).
- Construir un modelo de amenazas formal (DFD + STRIDE) y priorizar con CVSS v3.1.
- Ejecutar pruebas dinámicas controladas sobre la aplicación desplegada en entorno propio.
- Evaluar la configuración del despliegue en AWS contra el AWS Well-Architected Framework y el CIS Benchmark.
- Diseñar un plan de operación segura (NIST CSF 2.0) y una arquitectura endurecida.

2.3 Alcance y consideraciones éticas

Se deja constancia de que todas las pruebas realizadas se ejecutaron sobre entornos controlados, únicamente con fines académicos, y rigiéndonos estrictamente bajo los marcos establecidos por la Ley 21.459. Todas las pruebas se realizaron en un entorno de pruebas personal bajo una instancia propia, nunca sobre infraestructura ajena. Toda información sensible, como credenciales temporales propias o ajenas, no se publica y aparece censurada en las evidencias donde pudiera figurar.

Entorno de pruebas: backend en localhost:8000, frontend en localhost:5173, bucket archivacloud-p10-jz (us-east-1), credenciales temporales de AWS Academy (rol voclabs).

2.4 Marco metodológico

La auditoría se fundamenta en los siguientes marcos y estándares:

Marco / Estándar	Uso en el trabajo
OWASP Top 10 (2021)	Clasificación de vulnerabilidades web (frontend/lógica).

OWASP API Security Top 10 (2023)	Clasificación de vulnerabilidades del backend REST.
OWASP ASVS v4.0.3	Checklist de requisitos para validar controles defensivos.
MITRE ATT&CK (Cloud)	Mapeo de técnicas de ataque sobre AWS.
NIST CSF 2.0	Estructura del plan de mantención y operación segura.
ISO/IEC 27001:2022 + 27002	Selección y justificación de controles del Anexo A.
AWS Well-Architected (Security)	Auditoría del despliegue y la arquitectura cloud.
CIS AWS Foundations Benchmark v3.0	Benchmark de hardening de la cuenta AWS.
CVSS v3.1	Puntuación de severidad de cada vulnerabilidad.

2.5 Arquitectura de la aplicación auditada

Para el desarrollo de la aplicación el equipo TI utilizó un patrón de arquitectura llamado enlace temporal autorizado (URL prefirmada) la cual funciona de la siguiente forma :

1. el cliente solicita subir un archivo desde el navegador
2. El backend valida tipo/tamaño declarados y genera un enlace temporal a Amazon S3
3. El cliente sube el archivo directamente a S3 usando ese enlace sin pasar por el servidor

Gracias a esta arquitectura se reduce la carga del servidor, pero genera un punto crítico de seguridad: la subida directa desde el navegador a S3 (representada por la línea naranja en la Figura 1) elude cualquier control posterior del backend. Una vez que el enlace fue emitido, el servidor no puede intervenir ni verificar el contenido que finalmente se almacena.

El diagrama de flujo de datos de la Figura 1 ilustra los tres componentes de la aplicación y sus interacciones, destacando las fronteras de confianza entre el entorno del cliente, el servidor y la nube.

Diagrama de flujo de datos (DFD nivel 1) — ArchivaCloud

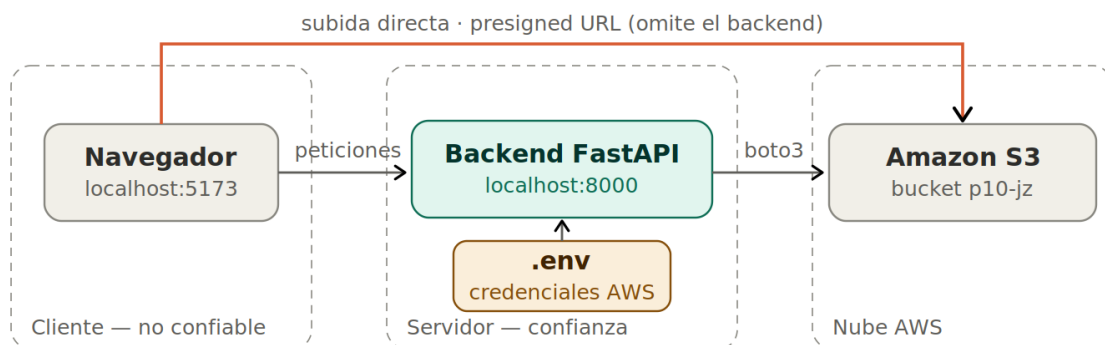


Figura 1 Diagrama de flujo de datos (DFD) nivel 1 de ArchivaCloud

El flujo crítico es la subida directa navegador→S3 mediante presigned URL, que cruza la frontera del servidor sin pasar por validación: el backend emite el permiso pero no controla el contenido que finalmente se sube.

3. Fase 1 — Análisis estático (SAST/SCA)

3.1 Metodología y herramientas

Se combinó revisión manual del código con cuatro herramientas automatizadas (dos SAST de Python, una SCA de JavaScript, una SCA de Python):

Herramienta	Comando	Resultado
Bandit	bandit -r app	0 hallazgos (111 LoC)
Semgrep (OSS)	semgrep --config=auto app	0 hallazgos (290 reglas)
npm audit	npm audit	0 vulnerabilidades
pip-audit	pip-audit	8 vuln. en 3 paquetes

3.2 Revisión manual del código

Conforme a lo indicado en el enunciado, se revisó el archivo .env.example incluido en el repositorio para verificar si contenía credenciales reales. Se confirmó que el archivo contiene únicamente valores de ejemplo (*placeholders*), sin claves de acceso reales, por lo que el control SEC-01 se cumple correctamente en este punto. Las credenciales reales residen únicamente en el archivo .env local, correctamente excluido del repositorio mediante .gitignore, por lo que no existe exposición de credenciales en el código publicado.

Vuln-001 Sin autenticación

La primera vulnerabilidad encontrada dentro del proyecto, concretamente dentro del backend, es que las funciones de los métodos GET, DELETE y POST no cuentan con autenticación, debido a la ausencia de un parámetro de autenticación, un decorador de verificación o algún elemento que indique que primero se debe validar la identidad del usuario en FastAPI. Debido a esta exclusión, cualquier petición que llegue al servidor es atendida sin preguntas. Esto fue comprobado y evidenciado mediante peticiones directas con curl, sin ningún tipo de credencial (Evidencia 12 y 14).

Evidencia relacionada con Análisis de código back end

- Evidencia 3 Código: endpoint DELETE (VULN-001/005)
- Evidencia 4 presigned-url + str(e) (VULN-002/004/005)
- Evidencia 5 Código: list_files (VULN-001/005)

Evidencia relacionada con salidas y scripts en CMD

- Evidencia 13 Path traversal: key con ../../ (VULN-002)
- Evidencia 15 html aceptado como PDF (VULN-004)

Vuln-004 Validación de tipo falsificable

La vulnerabilidad se debe a que el servidor valida el tipo de archivo basándose únicamente en lo declarado por el cliente, sin verificar el contenido real. Como se comprobó y evidenció en las pruebas, es posible subir un archivo HTML declarándolo como PDF, y el servidor no lo rechaza (Evidencia 15). La validación existe, pero se puede eludir enviando la petición directamente al servidor.

Evidencia relacionada con Análisis de código back end

- Evidencia 4 presigned-url + str(e) (VULN-002/004/005)

Evidencia relacionada con salidas, scripts en CMD y vista Navegador

- Evidencia 15 html aceptado como PDF (VULN-004)
- Evidencia 16 Archivo malicioso almacenado en la app (VULN-004)

Vuln-002 Path trasversal

El nombre del archivo se incorpora directamente a la ruta de almacenamiento sin ningún tipo de limpieza o verificación. Un atacante puede enviar un nombre con secuencias como ../../ para salir de la carpeta uploads/ prevista, como se demostró en las pruebas Evidencia 13 . En el contexto de Amazon S3, el impacto principal es la posibilidad de almacenar archivos fuera de la zona controlada y de sobrescribir archivos existentes.

Evidencia relacionada con Análisis de código back end

- Evidencia 2 Código: key=f"uploads/{fileName}" (VULN-002)

Evidencia relacionada con salidas y scripts en CMD

- Evidencia 13 Path traversal: key con ../../ (VULN-002)

Vuln-003 Sobreescritura

El servidor genera el enlace de subida para cualquier nombre de archivo sin verificar si ya existe un objeto con esa clave. Esto permite que un atacante suba un archivo con el mismo nombre que uno existente y lo reemplace sin ninguna advertencia. Como el versionado del bucket está deshabilitado (VULN-008), el archivo original se pierde de forma irreversible, como se demostró en las pruebas .

Evidencia relacionada con Análisis de código back end

- Evidencia 2 Código: key=f"uploads/{fileName}" (VULN-002)

Evidencia relacionada con salidas y scripts en CMD

- Evidencia 18 Sobreescritura confirmada con aws s3 cp (VULN-003)

Vuln-005 Fuga de información

Cuando ocurre un error interno, los tres puntos de acceso devuelven el mensaje de error completo al cliente sin filtrarlo. Esto expone detalles internos del sistema, como el nombre de las operaciones del servicio de almacenamiento utilizado. Como se demostró en las pruebas, un error provocado devuelve mensajes como 'An error occurred (400) when calling the DeleteObject operation', revelando que la aplicación usa Amazon S3 y el nombre exacto de la operación fallida .

Evidencia relacionada con Análisis de código back end

- Evidencia 3 Código: endpoint DELETE (VULN-001/005)
- Evidencia 4 presigned-url + str(e) (VULN-002/004/005)
- Evidencia 5 Código: list_files (VULN-001/005)

Evidencia relacionada con salidas y scripts en CMD

- Evidencia 17 Error crudo de boto3 al cliente (VULN-005)

3.3 Análisis automatizado e interpretación

- Bandit (Evidencia 1), Semgrep (Evidencia 10) y npm audit (Evidencia 9): 0 hallazgos.
- pip-audit (Evidencia 6): 8 vulnerabilidades en 3 paquetes.

Lectura: el código propio no presenta errores de programación reconocibles por las herramientas, y las dependencias de JavaScript no tienen vulnerabilidades conocidas, pero las de Python sí. Las fallas de mayor impacto de esta aplicación son de diseño y de configuración, que el análisis automático no detecta: por eso el análisis del código es necesario, pero no suficiente, y se complementa con la revisión manual (Fase 1) y las pruebas en ejecución (Fase 3). Las ocho vulnerabilidades detectadas por pip Audit se detallan en la sección 3.4

Salida de prueba de ejecución de análisis automático semgrep,npm Audit,pip Audit y bandit

- Evidencia 10 Semgrep: 0 hallazgos (Fase 1)
- Evidencia 9 npm audit: 0 vulnerabilidades (Fase 1)
- Evidencia 6 pip-audit: 8 vuln. en 3 paquetes (VULN-006/007)
- Evidencia 1 Bandit: sin hallazgos (Fase 1)

3.4 Hallazgos de dependencias (SCA)

El análisis con pip-audit identificó un total de 8 vulnerabilidades conocidas, distribuidas en 3 paquetes del entorno Python del backend. La siguiente tabla detalla cada paquete afectado:

Paquete	Versión	CVE / Aviso (CVSS)	Fix
starlette	1.2.1	CVE-2026-54283 (7.5), CVE-2026-54282	1.3.1
msgpack	1.1.2	GHSA-6v7p-g79w-8964 (7.5)	1.2.1
pip	25.1.1	5 CVE (PYSEC-2026-196, CVE-2025-8869, +3) — herramienta, no runtime	26.1.2

Nota: las 5 CVE de pip corresponden a la herramienta de instalación (no se ejecuta en producción), por lo que se documentan como riesgo bajo y no como ficha completa. A diferencia de starlette y msgpack, que sí afectan directamente el funcionamiento de la aplicación, por lo que cada una se documenta de forma propia.

3.5 Discrepancia entre la documentación de seguridad y la implementación real

El equipo desarrollador entregó un documento (SEGURIDAD.MD) que declara la implementación de 10 controles de seguridad obligatorios (SEC-01 a SEC-10) al realizar validaciones de implementación sobre esos controles en el código fuente real y con las pruebas dinámicas de esta auditoría se identificó que al menos 4 de esos controles no están correctamente implementados a pesar de estar documentados como cumplidos:

- **SEC-03 Validación de entrada:** se declara una función `clean_filename` que sanitiza los nombres de los archivos subidos. El código real no contiene la función declarada; el nombre

del cliente se incorpora directamente a la ruta de almacenamiento sin sanitización (VULN-002).

- **SEC-05 IAM mínimo privilegio:** se declara una política JSON que restringe boto3 a 4 operaciones específicas. La política nunca se aplica de forma efectiva; el despliegue auditado opera con el rol amplio de AWS Academy (voclabs), sin la restricción declarada (VULN-012).
- **SEC-07 Errores sin información sensible:** se declara que los errores devuelven un mensaje genérico con código HTTP 502. El código real reenvía el error interno completo de boto3 al cliente con código HTTP 500, exponiendo información sensible (VULN-005).
- **SEC-09 Escaneo de dependencias:** se declara un entorno libre de vulnerabilidades críticas. La ejecución de pip-audit revela 8 vulnerabilidades en 3 paquetes, incluidas 2 de severidad alta en starlette, componente central del framework (VULN-006).

Esta discrepancia constituye un hallazgo de proceso, independiente de los hallazgos técnicos anteriores: la documentación de seguridad no refleja el estado real del sistema.

3.6 Tabla resumen de hallazgos

La siguiente tabla consolida los 13 hallazgos identificados en la auditoría. Los primeros 7 corresponden a la revisión manual del código y a la ejecución de los análisis automáticos de dependencias, desarrollados en este capítulo; los 6 restantes corresponden a la configuración de la nube, que se detalla en la Fase 4 (capítulo 6). Cada hallazgo cuenta con su ficha completa en el Catálogo de vulnerabilidades (capítulo 10).

ID	Hallazgo	OWASP	CWE/CVE	Severidad
VULN-001	Ausencia de autenticación en los 3 endpoints	A01 / API2	CWE-306	Crítico
VULN-002	Path/key traversal en parámetro fileName	A01 / API1	CWE-22	Alto
VULN-003	Sobrescritura de objetos sin control de propiedad	A01 / API1	CWE-639	Alto
VULN-004	Validación de tipo de archivo falsificable	A04	CWE-434	Medio
VULN-005	Fuga de información en mensajes de error	A09 / API8	CWE-209	Medio
VULN-006	Dependencia vulnerable: starlette 1.2.1	A06	CVE-2026-54283	Alto
VULN-007	Dependencia vulnerable: msgpack 1.1.2	A06	GHSA-6v7p-...	Medio
VULN-008	Versioning del bucket deshabilitado	A04	CWE-693	Alto
VULN-009	Sin server access logging en el bucket	A09	CWE-778	Medio
VULN-010	Cifrado SSE-S3 en lugar de SSE-KMS/CMK	A02	CWE-311	Medio
VULN-011	Sin bucket policy que fuerce TLS	A05	CWE-319	Medio
VULN-012	Credenciales en .env / IAM sin mínimo privilegio	A05 / A07	CWE-522	Alto
VULN-013	Validación solo del lado cliente (eludible)	A04	CWE-602	Medio

Los hallazgos provienen de tres orígenes distintos: fallas de lógica y diseño en el código propio, dependencias con vulnerabilidades conocidas, y configuraciones inseguras en el despliegue. Las vulnerabilidades de dependencias sí fueron detectadas por el análisis automático de composición (pip-audit); en cambio, ninguna de las fallas de código ni de configuración fue identificada por las herramientas de análisis estático (Bandit, Semgrep), lo que confirma la conclusión de la sección 3.3: dichas herramientas son necesarias, pero no suficiente

4. Fase 2 — Modelado de amenazas

4.1 Diagrama de flujo de datos y fronteras de confianza

El modelado de amenazas parte del diagrama de flujo de datos (DFD) presentado en la (sección 2.5), que representa cómo se mueve la información dentro de la aplicación. El diagrama identifica los siguientes elementos:

1. Entidad externa Navegador: el usuario que interactúa con la aplicación a través del navegador. Se considera externo y no confiable, ya que los datos que envía pueden ser manipulados.
2. Proceso Backend: la aplicación FastAPI que recibe las peticiones, las valida y genera los enlaces de subida.
3. Almacenes de datos Bucket S3 y archivo .env: el bucket S3 guarda los documentos; el archivo .env guarda las credenciales de acceso a la nube.
4. Flujos de datos: las interacciones entre los elementos, como las peticiones del navegador al backend, las llamadas del backend a S3, y la subida directa del navegador a S3.

El diagrama define además tres fronteras de confianza, representadas con líneas punteadas, que separan zonas con distinto nivel de seguridad:

1. Cliente (no confiable): el entorno del navegador, fuera del control de la aplicación.
2. Servidor (de confianza): el entorno donde se ejecuta el backend y se almacenan las credenciales.
3. Nube AWS: el entorno de almacenamiento gestionado por Amazon.

Las fronteras de confianza son relevantes porque las amenazas de seguridad suelen aparecer precisamente donde los datos cruzan de una zona a otra. En esta aplicación, el cruce más crítico es la subida directa desde el navegador (zona no confiable) hacia S3 (nube), ya que omite el control del servidor. El análisis STRIDE de la sección siguiente se aplica sobre cada uno de estos elementos y fronteras.

- Figura 1 Diagrama de flujo de datos (DFD) nivel 1 de ArchivaCloud

4.2 Matriz STRIDE

Aplicación de STRIDE a cada elemento del DFD, cruzada con los hallazgos identificados:

Elemento	STRIDE	Amenaza	Hallazgo
Entidad: Navegador	Suplantación (S)	Sin autenticación: cualquiera se hace pasar por usuario válido.	VULN-001
Entidad: Navegador	Manipulación (T)	El cliente altera el nombre, tipo y tamaño del archivo enviados.	VULN-002, 004, 013
Entidad: Navegador	Repudio (R)	Acciones sin identidad ni registro: no atribuibles.	VULN-001, 009
Proceso: Backend	Elevación de privilegios (E)	Funciones críticas accesibles sin control de acceso.	VULN-001
Proceso: Backend	Divulgación de información (I)	Los mensajes de error exponen detalles internos del sistema.	VULN-005
Proceso: Backend	Denegación de servicio (D)	Sin límite de peticiones; una dependencia agrava el riesgo.	VULN-006 (obs.)
Almacén: Bucket S3	Manipulación (T)	Recorrido de rutas y sobrescritura; sin versionado es irreversible.	VULN-002, 003, 008
Almacén: Bucket S3	Divulgación de información (I)	Sin registro de accesos; cifrado sin clave propia.	VULN-009, 010
Almacén: .env	Divulgación de información (I)	Credenciales de AWS en texto plano en el servidor.	VULN-012
Almacén: .env	Elevación de privilegios (E)	Filtrar el .env compromete toda la cuenta.	VULN-012
Flujo: Nav→S3 directo	Manipulación (T)	Subida sin validación del servidor (omite el backend).	VULN-002, 004
Flujo: Backend↔S3	Divulgación de información (I)	Sin regla que obligue a usar cifrado (TLS) en el transporte.	VULN-011

4.3 Priorización (CVSS v3.1)

Para priorizar la remediación, los 13 hallazgos se ordenan según su impacto estimado para el negocio, no estrictamente por su puntaje CVSS. Esto se debe a que la mayoría son vulnerabilidades específicas del diseño y la implementación del proyecto (y no librerías con un CVE oficial), por lo que sus puntajes son estimaciones del auditor según el vector de cada ficha; solo VULN-006 y VULN-007 usan el puntaje oficial del NVD, por corresponder a dependencias conocidas. El puntaje CVSS se incluye como referencia, pero el orden prioriza el daño real que cada hallazgo representa para CloudConta SpA.

Prioridad	Hallazgo	CVSS	Severidad
1	VULN-001: Sin autenticación	9.1	Critico

2	VULN-012: Credenciales en .env / IAM amplio	7.0	Alto
3	VULN-003: Sobrescritura sin control	7.5	Alto
4	VULN-002 — Path traversal	7.5	Alto
5	VULN-008 — Sin versionado	6.5	Alto
6	VULN-006 — Dependencia starlette	7.5	Alto
7	VULN-007 — Dependencia msgpack	7.5	Alto
8	VULN-005 — Fuga de información en errores	5.3	Medio
9	VULN-004 — Tipo de archivo falsificable	5.4	Medio
10	VULN-011 — Sin política TLS	5.9	Medio
11	VULN-013 — Validación solo del lado cliente	5.3	Medio
12	VULN-010 — Cifrado SSE-S3 sin CMK	4.2	Medio
13	VULN-009 — Sin registro de accesos	3.2	Medio

El orden refleja el daño potencial para el negocio: encabezan los hallazgos que exponen o comprometen directamente los documentos de los clientes (acceso sin autenticación, compromiso de credenciales, manipulación de archivos), seguidos de los que afectan la recuperación y la integridad (versionado, dependencias). Los hallazgos de severidad media, aunque no críticos por sí solos, refuerzan el riesgo general y se abordan en el plan de mantención (Fase 5).

Por ende, se entiende que las cinco amenazas con prioridad son: las 5 primeras de la lista VULN-001 (ausencia de autenticación), VULN-012 (credenciales y privilegios excesivos), VULN-003 (sobrescritura de archivos), VULN-002 (path traversal) y VULN-008 (falta de versionado). Estas concentran el mayor impacto potencial sobre la confidencialidad, integridad de la información e impacto en el negocio y deben priorizarse en la remediación.

4.4 Mapeo MITRE ATT&CK for Cloud

Hallazgo	Técnica MITRE ATT&CK	Descripción
VULN-001	T1530 — Data from Cloud Storage	Lectura de objetos del bucket sin credenciales.
VULN-001, 012	T1078.004 — Valid Accounts: Cloud Accounts	Uso de credenciales/acceso sin control de identidad.
VULN-002, 003	T1565.001 — Stored Data Manipulation	Manipulación y sobrescritura de objetos almacenados.
VULN-012	T1552.001 — Unsecured Credentials: Credentials In Files	Credenciales en archivo .env.
VULN-006	T1499 — Endpoint Denial of Service	Agotamiento de recursos por parseo de formularios.
VULN-009	T1562.008 — Impair Defenses: Disable/Modify Cloud Logs	Ausencia de registro dificulta detección (gap defensivo).

4.5 Caso de abuso encadenado

Escenario que combina varias debilidades:

Un atacante anónimo (sin credenciales, VULN-001) consulta GET /api/files y obtiene los nombres de todos los documentos. Identifica un archivo crítico y, aprovechando la ausencia de control de propiedad y de saneamiento del nombre (VULN-002/003), solicita una URL prefirmada para esa misma ubicación y sube contenido manipulado, sobrescribiendo el original. Como el versionado está deshabilitado (VULN-008), la pérdida es irreversible y, sin registro de accesos (VULN-009), el incidente no deja rastro. Adicionalmente, podría subir un archivo HTML malicioso declarándolo PDF (VULN-004) para un eventual ataque XSS (ejecución de código en el navegador de la víctima)

5. Fase 3 — Pruebas dinámicas (DAST / pentesting controlado)

5.1 Entorno y reglas de ejecución

Todas las pruebas dinámicas se ejecutaron sobre la instancia propia desplegada para esta auditoría: el backend en localhost:8000, el frontend en localhost:5173 y el bucket de pruebas archivacloud-p10-jz, conforme al alcance ético declarado en la sección 2.3. Nunca se ejecutaron pruebas sobre infraestructura ajena.

La herramienta principal fue curl, utilizada para enviar peticiones directas al servidor y simular el comportamiento de un atacante que omite el navegador. Para la prueba de CORS se utilizó además una página HTML local, ejecutada desde un origen distinto al autorizado.

Las pruebas que implican consumo elevado de recursos (denegación de servicio y subida masiva de archivos) se documentaron de forma teórica y no se ejecutaron, con el fin de no agotar los recursos del laboratorio AWS Academy. Su análisis se basa en la revisión del código y de las dependencias.

Como se mencionó con anterioridad para las pruebas se optó por curl en lugar de herramientas como Burp suite o OWASP ZAP ya que los vectores identificados (ausencia de autenticación, manipulación de parámetros, path traversal y fuga de errores) se demuestran de forma más precisa y reproducible enviando peticiones controladas directamente al servidor. curl permite construir cada payload de manera explícita y documentar el comando exacto, lo que facilita la reproducción de cada prueba por parte de un tercero.

5.2 Resultados por vector

A continuación, se describe cada vector de ataque probado, con su resultado y la evidencia correspondiente. El detalle reproducible (comandos y respuestas completas) se encuentra en el Anexo A.

- Sin autenticación (lectura): se envió una petición GET al endpoint /api/files sin ninguna credencial. El servidor respondió con la lista completa de archivos almacenados, confirmando que cualquier persona puede consultar el contenido del bucket sin identificarse (Evidencia 12, VULN-001).

Evidencia 12 GET /api/files sin token (VULN-001)

- Sin autenticación (borrado): se envió una petición DELETE sobre un archivo existente, también sin credenciales. El servidor eliminó el objeto y respondió "Archivo eliminado", confirmando que cualquier persona puede borrar documentos sin autorización (Evidencia 11, VULN-001).

Evidencia 11 DELETE sobre inexistente → "Archivo eliminado" (observación)

- Path traversal: se solicitó un enlace de subida enviando como nombre de archivo la secuencia "../../../escape/test.pdf". El servidor generó la ruta uploads/../../../escape/test.pdf sin sanitizarla, confirmando que el nombre del cliente se incorpora sin verificación y permite salir de la carpeta prevista (Evidencia 13, VULN-002).

Evidencia 13 Path traversal: key con ../../ (VULN-002)

- Sobrescritura: se subió un archivo con el mismo nombre que uno existente. El servidor generó el enlace sin verificar la existencia previa, y el objeto original fue reemplazado por el

del atacante, confirmado mediante la lectura posterior del archivo en S3 (Evidencia 18, VULN-003).

Evidencia 18 Sobrescritura confirmada con aws s3 cp (VULN-003)

- Tipo de archivo falsificable: se solicitó un enlace de subida para un archivo .html declarándolo como application/pdf. El servidor lo aceptó sin rechazarlo, confirmando que la validación se basa en el tipo declarado por el cliente y no en el contenido real (Figuras 15 y 16, VULN-004).

Evidencia 16 Archivo malicioso almacenado en la app (VULN-004)

Evidencia 15 html aceptado como PDF (VULN-004)

- Fuga de información en errores: se provocó un error enviando una clave inválida en una petición DELETE. El servidor devolvió el mensaje crudo de boto3 ("An error occurred (400) when calling the DeleteObject operation"), exponiendo detalles internos del sistema al cliente (Evidencia 17, VULN-005).

Evidencia 17 Error crudo de boto3 al cliente (VULN-005)

- CORS (control correcto): se intentó acceder a la API desde un origen no autorizado mediante una página HTML local. El navegador bloqueó la petición, confirmando que la restricción CORS está correctamente configurada. Este vector no constituye una vulnerabilidad, sino la verificación de un control que funciona (Evidencia 19).

Evidencia 19 CORS: origen externo bloqueado (control correcto)

Prueba	Resultado	Severidad	Evidencia
Sin autenticación (lectura)	GET /api/files devuelve la lista sin token.	Crítico	Evidencia 12 GET /api/files sin token (VULN-001)
Sin autenticación (borrado)	DELETE elimina objetos sin token.	Crítico	Evidencia 14 DELETE sin token (VULN-001)
Path traversal	Key resultante: uploads/../../escape/test.pdf	Alto	Evidencia 13 Path traversal: key con ../../ (VULN-002)
Sobrescritura	Objeto de la víctima reemplazado por el del atacante.	Alto	Evidencia 18 Sobrescritura confirmada con aws s3 cp (VULN-003)
Tipo falsificable	.html aceptado declarándolo application/pdf.	Medio	Evidencia 15 html aceptado como PDF (VULN-004) Evidencia 16 Archivo malicioso almacenado en la app (VULN-004)
Fuga de info en errores	Error crudo de boto3 (DeleteObject) al cliente.	Medio	Evidencia 17 Error crudo de boto3 al cliente (VULN-005)

CORS (control correcto)	Origen externo bloqueado por el navegador.	Informativo	Evidencia 19 CORS: origen externo bloqueado (control correcto)
-------------------------	--	--------------------	--

5.3 Síntesis de pruebas

De los siete vectores probados, seis confirmaron vulnerabilidades reales (VULN-001 a 005, más la sobrescritura) y uno (CORS) confirmó un control bien implementado. Esto comprueba en la práctica lo que se había anticipado en el modelo de amenazas (Fase 2): las amenazas de la matriz STRIDE no son solo teóricas, sino explotables en la práctica.

Estas pruebas se hicieron sobre la aplicación funcionando. Para lograrlo, fue necesario corregir de forma local un error que impedía ejecutarla, trabajando sobre una rama aparte (fix-local-auditoria) y dejando la versión original sin tocar como evidencia. Esto permitió probar el comportamiento real del sistema y no solo revisar el código.

En conjunto, el análisis del código (Fase 1) y las pruebas en ejecución (Fase 3) confirman cada hallazgo desde dos ángulos distintos, lo que le da mayor solidez a los resultados.

6. Fase 4 — Auditoría del despliegue en la nube

6.1 IAM y credenciales

La aplicación accede a Amazon S3 mediante credenciales almacenadas en un archivo `.env` en el servidor. En el entorno auditado son credenciales temporales del rol `voclabs` de AWS Academy, como se verificó al consultar la identidad activa (Evidencia 8, VULN-012).

Es importante aclarar que el archivo `.env` está correctamente excluido del repositorio mediante `.gitignore` (control SEC-01), por lo que las credenciales no se encuentran expuestas en el código. El hallazgo no se refiere a una filtración actual, sino al método de almacenamiento: guardar credenciales en un archivo de texto es un anti-patrón, ya que cualquier acceso indebido al servidor o un error de configuración bastaría para comprometerlas. La práctica recomendada es entregar los permisos directamente al servicio mediante un rol asociado (por ejemplo, un `instance profile`), evitando que exista una credencial almacenada.

Adicionalmente, el documento `seguridad.txt` declara una política IAM de mínimo privilegio que nunca se aplica: el despliegue opera con el rol `voclabs`, de permisos amplios. Dicha política existe solo como texto de referencia, no como un control real.

6.2 Configuración del bucket S3

La auditoría de la configuración del bucket `archivacloud-p10-jz` se realizó mediante comandos de la AWS CLI, cuyos resultados se muestran en la Evidencia 7. Se evaluaron los siguientes aspectos:

- Cifrado en reposo: el bucket cifra los archivos con SSE-S3 (clave gestionada por AWS). Es un control presente, pero limitado: para datos sensibles se recomienda SSE-KMS con clave propia (CMK), que permite controlar la rotación y el uso de la clave (VULN-010).
- Versionado: está deshabilitado. Sin versionado, no es posible recuperar un archivo después de que se sobrescriba o se elimine, lo que agrava la sobrescritura demostrada en la Fase 3 (VULN-008).
- Registro de accesos: no está activado. No existe ninguna bitácora de quién accede o modifica los archivos, lo que impide investigar un incidente (VULN-009).
- Política de bucket / TLS: no existe ninguna política que obligue a usar conexiones cifradas (HTTPS). Sin ella, no se garantiza que el acceso al bucket sea siempre seguro (VULN-011).
- Gestión de retención (lifecycle): no hay reglas de retención ni de limpieza de archivos antiguos o subidas incompletas.
- Block Public Access: está activado en su totalidad. Este es un control correctamente aplicado: el bucket no es accesible públicamente desde internet.

6.3 CORS del bucket

La configuración CORS del bucket restringe correctamente el origen permitido a `localhost:5173`, evitando el uso de comodines en ese campo. Sin embargo, habilita los métodos GET, POST, PUT y DELETE, además de permitir todas las cabeceras (`headers *`).

Es necesario aclarar que esta configuración CORS aplica únicamente a las peticiones que el navegador envía directamente a S3, es decir, la subida mediante URL prefirmada. Las operaciones de listado y borrado de la aplicación no pasan por esta vía: las realiza el backend mediante `boto3`, una comunicación de servidor a servidor que no está sujeta a CORS. Por lo tanto, para la comunicación directa navegador-S3 solo se requiere el método PUT (subir), y eventualmente GET (leer). Habilitar también POST y DELETE, junto con todas las cabeceras, otorga más permisos de los necesarios.

6.4 Auditoría contra CIS AWS Foundations Benchmark v3.0

Para complementar la auditoría de configuración, se evaluó el despliegue contra el CIS AWS Foundations Benchmark v3.0, un estándar reconocido de buenas prácticas para asegurar cuentas de AWS. El benchmark completo contiene 37 controles distribuidos en cinco secciones (IAM, almacenamiento, registro, monitoreo y red). Para esta auditoría se seleccionaron los 10 controles directamente aplicables a una aplicación de almacenamiento basada en Amazon S3 y su gestión de identidades, omitiendo aquellos que no corresponden al alcance del sistema evaluado (por ejemplo, los relativos a RDS, EC2 o la cuenta root). La siguiente tabla resume el estado de cada control y su recomendación:

Control CIS	Descripción	Estado	Recomendación
CIS 2.1.1	S3: bucket policy deniega peticiones HTTP (exige TLS)	No cumple	No existe policy; agregar Deny si <code>aws:SecureTransport=false</code> .
CIS 2.1.2	S3: MFA Delete habilitado	No cumple	Habilitar MFA Delete (requiere versionado previo).
CIS 2.1.4	S3: Block Public Access configurado	Cumple	Control correctamente aplicado (4/4 flags).
CIS 1.16	IAM: políticas con privilegios *:.* no asignadas	No cumple	El rol <code>voclabs</code> tiene permisos amplios; aplicar mínimo privilegio.
CIS 1.18	IAM: uso de instance roles para acceso desde instancias	No cumple	Credenciales en <code>.env</code> ; usar instance profile en lugar de archivo.
CIS 1.14	IAM: rotación de access keys cada 90 días o menos	No aplica	Credenciales temporales del lab; en producción, establecer rotación.
CIS 3.1	CloudTrail habilitado en todas las regiones	No cumple	Sin trazabilidad de API; habilitar CloudTrail.
CIS 3.8	S3: logging de objetos para eventos de escritura	No cumple	Sin registro de accesos; habilitar object-level logging.
CIS 3.9	S3: logging de objetos para eventos de lectura	No cumple	Sin registro de accesos; habilitar object-level logging.
CIS 4.16	AWS Security Hub habilitado	No cumple	Sin monitoreo continuo; habilitar en producción.

De los 10 controles evaluados, solo uno se cumple completamente (Block Public Access, control 2.1.4). De los nueve restantes, uno no aplica en este caso, ya que las credenciales del laboratorio rotan automáticamente en cada despliegue por diseño de AWS Academy. Los ocho controles restantes no se cumplen, lo que evidencia que el despliegue se realizó sin aplicar las medidas de endurecimiento y seguridad recomendadas por el estándar. Estas brechas se abordan en la arquitectura objetivo propuesta en la Fase 6

6.5 Evaluación contra el AWS Well-Architected Framework

Además del benchmark CIS, los hallazgos se contrastaron con los principios de diseño del Pilar de Seguridad del AWS Well-Architected Framework:

Principio de diseño	Estado de la aplicación	Hallazgo relacionado
Identidad solida	No cumplido: sin autenticación ni mínimo privilegio	VULN-001, VULN-012
Trazabilidad	No cumplido: sin registro de accesos ni CloudTrail	VULN-009
Seguridad en todas las capas	Parcial: falta WAF, validación servidor y política TLS	VULN-004, VULN-011, VULN-013
Automatizar buenas practicas	No cumplido: configuración manual, sin IaC	(se corrige en Fase 6)
Proteger datos en tránsito y reposo	Parcial: cifrado SSE-S3 sin CMK; TLS no forzado	VULN-010, VULN-011
Mantener a las personas lejos de los datos	No cumplido: acceso directo sin control	VULN-001
Prepararse para incidentes	No cumplido: sin plan ni detección	(se aborda en Fase 5)

La evaluación confirma que la aplicación no satisface la mayoría de los principios del pilar, coincidiendo con los hallazgos de las fases anteriores. La arquitectura objetivo (Fase 6) se diseñó precisamente para cubrir estos principios.

7. Fase 5 — Plan de operación y mantención segura (NIST CSF 2.0)

Esta fase propone un plan de operación y mantención continua de la seguridad, estructurado según las seis funciones del marco NIST Cybersecurity Framework 2.0. El objetivo es que las correcciones aplicadas no sean puntuales, sino parte de un proceso permanente que mantenga el sistema seguro a lo largo del tiempo.

7.1 Gobernar

Se deben definir los roles y responsabilidades de seguridad sobre la aplicación, estableciendo quién es responsable de la gestión de credenciales, del monitoreo y de la respuesta a incidentes. Se recomienda formalizar una política de seguridad que incluya el manejo de datos de clientes y el cumplimiento de la Ley 19.628 sobre protección de la vida privada (y su actualización, la Ley 21.719), dado que la aplicación gestiona documentos que pueden contener datos personales. Esta función también establece la obligación de auditar periódicamente el sistema, en lugar de confiar únicamente en la autoevaluación del equipo de desarrollo.

7.2 Identificar

Se debe mantener un inventario actualizado de los activos en la nube (bucket S3, credenciales IAM, servicios asociados) y una clasificación de los documentos según su sensibilidad, para aplicar controles proporcionales al riesgo. Asimismo, se debe inventariar y monitorear las dependencias del proyecto (como starlette y msgpack, detectadas en la Fase 1) para reaccionar ante nuevas vulnerabilidades conocidas.

7.3 Proteger

Esta función agrupa las medidas preventivas que corrigen los hallazgos de la auditoría:

- Implementar autenticación y autorización en la aplicación (corrige VULN-001).
- Migrar las credenciales desde el archivo .env hacia un gestor seguro de secretos como AWS Secrets Manager, y aplicar el principio de mínimo privilegio en IAM (corrige VULN-012).

- Habilitar la autenticación multifactor (MFA) en las cuentas con acceso a la nube
- Migrar el cifrado del almacenamiento a una clave propia en AWS KMS (CMK), para controlar su rotación y uso (corrige VULN-010).
- Reforzar la validación de entradas del backend y la gestión de errores (corrige VULN-002, 004 y 005).
- Mantener un inventario de componentes de software (SBOM) y actualizar las dependencias vulnerables (corrige VULN-006 y 007).

7.4 Detectar

Se debe implementar un registro centralizado de actividad que permita detectar accesos o acciones sospechosas. Como medida base, habilitar el registro de accesos del bucket y CloudTrail (corrige VULN-009), y definir métricas y alarmas en Amazon CloudWatch para eventos anómalos (por ejemplo, un volumen inusual de borrados o accesos no autenticados). En un escenario de mayor madurez, estos registros pueden integrarse con una herramienta de monitoreo de seguridad (SIEM) para su análisis continuo.

7.5 Responder

Se debe contar con un plan de respuesta a incidentes que defina los pasos a seguir ante un evento de seguridad, como el compromiso de credenciales o el acceso no autorizado a documentos. El plan debe incluir los procedimientos de contención (por ejemplo, revocar credenciales comprometidas), las vías de comunicación y escalamiento, y una asignación clara de responsabilidades mediante la matriz RACI presentada en la sección 7.7.

7.6 Recuperar

Se deben establecer mecanismos que permitan restaurar el sistema y los datos tras un incidente. Esto incluye habilitar el versionado del bucket para recuperar archivos sobrescritos o eliminados (corrige VULN-008), definir una política de respaldos periódicos, y documentar un plan de continuidad con objetivos claros de recuperación: el RPO (cuánta información se acepta perder como máximo) y el RTO (en cuánto tiempo debe restaurarse el servicio).

7.7 Matriz RACI y cronograma

Como punto 7.7 se presenta la matriz RACI, en la cual se detallan los responsables sugeridos. En este caso particular, como el trabajo es de carácter individual, se detallan roles funcionales y no personas específicas.

Actividad	R	A	C	I
Rotación de credenciales y Gestión de secretos	Administrador Cloud	Responsable de seguridad	DevOps	Gerencia
Revisión de logs / alertas	Analista de seguridad	Responsable de seguridad	Administrador Cloud	X
Parcheo de dependencias	Desarrollador	Responsable de seguridad	DevOps	X
Pruebas de recuperación (backup/restore)	Administrador cloud	Responsable de seguridad	Desarrollador	Gerencia

Para dar continuidad a las tareas detalladas, se sugiere un cronograma de tareas cíclicas con su frecuencia recomendada.

Frecuencia	Tarea
Diaria	Revisión de alertas de seguridad y registros de acceso.
Semanal	Verificación de dependencias vulnerables (pip-audit / npm audit).
Mensual	Revisión de permisos IAM y rotación de secretos.
Trimestral	Prueba de recuperación de respaldos y revisión de la configuración del bucket.
Anual	Auditoría de seguridad completa (repetir este proceso).

8. Fase 6 — Propuesta de arquitectura segura

8.1 Diagrama de arquitectura objetivo

La Figura 2 presenta la arquitectura objetivo propuesta, una versión endurecida de ArchivaCloud que corrige los hallazgos identificados en la auditoría. A diferencia de la arquitectura original, esta incorpora controles de seguridad en cada capa: una capa de borde que filtra el tráfico, autenticación obligatoria antes de acceder a la aplicación, gestión segura de credenciales, almacenamiento cifrado con clave propia y registro de toda la actividad. El flujo general se mantiene (el usuario interactúa con la aplicación y los archivos se almacenan en S3), pero ahora cada paso pasa por un control que en la versión auditada estaba ausente.

Arquitectura objetivo endurecida — ArchivaCloud

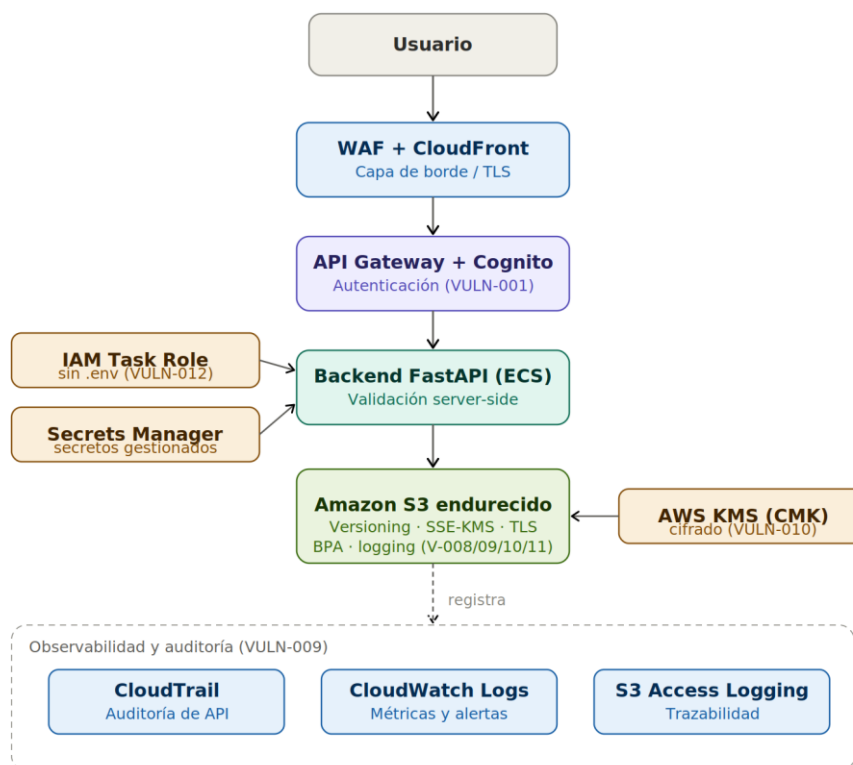


Figura 2 Arquitectura objetivo endurecida de ArchivaCloud

8.2 Decisiones y justificación

Cada componente de la arquitectura objetivo responde a un hallazgo concreto de la auditoría:

- API Gateway + Amazon Cognito (o ALB + JWT): incorpora autenticación y autorización obligatorias antes de acceder a los endpoints, de modo que ningún usuario anónimo pueda listar, subir o eliminar archivos. Corrige el hallazgo crítico VULN-001.
- IAM Task Role + AWS Secrets Manager: la aplicación obtiene sus permisos mediante un rol asociado al servicio de cómputo, eliminando las credenciales del archivo .env y aplicando el principio de mínimo privilegio. Corrige VULN-012.
- Cifrado SSE-KMS con clave propia (CMK): reemplaza la clave gestionada por AWS por una clave bajo control de la organización, con rotación y trazabilidad de uso. Corrige VULN-010.
- Bucket policy con TLS obligatorio: una política que rechaza las conexiones no cifradas (aws:SecureTransport=false) garantiza que todo acceso al bucket use HTTPS. Corrige VULN-011.
- Versionado del bucket: conserva versiones anteriores de cada archivo, permitiendo recuperar documentos sobrescritos o eliminados. Corrige VULN-008.
- CloudTrail + registro de accesos: registra toda la actividad sobre el bucket y la API, habilitando la detección e investigación de incidentes. Corrige VULN-009.
- WAF y VPC Endpoint para S3: el WAF filtra patrones de ataque conocidos en la capa de borde, y el VPC Endpoint permite que el backend acceda a S3 a través de la red interna de AWS, sin exponer el tráfico a internet. Refuerzan la seguridad general más allá de los hallazgos puntuales.

8.3 Infraestructura como código (IaC)

Para evitar que estos controles dependan de una configuración manual la arquitectura objetivo se define mediante Infraestructura como Código, usando Terraform. Este enfoque garantiza que el entorno siempre se despliegue con los controles de seguridad aplicados, de forma reproducible y verificable.

El código completo se incluye en el Anexo C (archivo main.tf) y corrige seis hallazgos de forma automática: versionado (VULN-008), cifrado con clave propia KMS (VULN-010), política TLS (VULN-011), Block Public Access, registro de accesos (VULN-009) y rol IAM de mínimo privilegio (VULN-012).

A continuación, se muestran los fragmentos más relevantes:

1.

```
# Versionado habilitado (corrige VULN-008)
resource "aws_s3_bucket_versioning" "archiva" {
  bucket = aws_s3_bucket.archiva.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

2. # Política que deniega conexiones sin TLS (corrige VULN-011)

```
resource "aws_s3_bucket_policy" "archiva" {
  bucket = aws_s3_bucket.archiva.id
  policy = jsonencode({
    Statement = [{
      Effect = "Deny"
      Principal = "*"
      Action = "s3:*"
    }]
  })
}
```



```

        Resource = ["${aws_s3_bucket.archiva.arn}/*"]
        Condition = { Bool = { "aws:SecureTransport" = "false" } }
    }}
})
}

3. # Rol IAM de mínimo privilegio: solo 4 operaciones S3 (corrige VULN-012)
resource "aws_iam_role_policy" "archiva_app" {
    name = "archivacloud-s3-min"
    role = aws_iam_role.archiva_app.id
    policy = jsonencode({
        Statement = [{
            Effect = "Allow"
            Action = ["s3:PutObject", "s3:GetObject", "s3:DeleteObject", "s3:ListBucket"]
            Resource = [aws_s3_bucket.archiva.arn, "${aws_s3_bucket.archiva.arn}/*"]
        }]
    })
}

```

Estos fragmentos ilustran cómo cada control queda definido de forma explícita y obligatoria en el código, a diferencia del despliegue auditado, donde la seguridad dependía de configuraciones manuales que no se aplicaron.

Código completo entregado como documento aparte tal como se pide en la bitácora

- [Anexo C — Código IaC](#)

8.4 Estimación de costos AWS

Se estimó el costo mensual de operar la arquitectura objetivo en la región us-east-1, bajo un escenario de uso bajo. Los valores son aproximados, basados en las tarifas públicas de AWS, y fueron establecidos con la AWS Pricing Calculator.

Servicio	Costo mensual estimado (USD)
Amazon S3 (almacenamiento + solicitudes)	0,30
AWS KMS (clave propia)	1,10
CloudTrail (data events)	1,50
CloudWatch Logs	1,00
Amazon Cognito (dentro de capa gratuita)	0,00
API Gateway	0,18
AWS WAF	6,00
Amazon CloudFront	1,00
Cómputo backend (ECS Fargate)	9,00
Total estimado	21 USD/mes

Esta estimación corresponde a un escenario de uso bajo y tiene carácter referencial; el costo real puede variar aproximadamente entre 15 y 30 USD mensuales según el volumen de tráfico, las horas de cómputo y la transferencia de datos. Los servicios de cómputo (Fargate) y el WAF son los componentes de mayor variabilidad, y podrían sustituirse por alternativas más económicas (por ejemplo, una instancia de capa gratuita para el backend). En cambio, los controles esenciales de seguridad (KMS, CloudTrail, registro de accesos) representan un costo bajo frente al valor de protección que aportan, por lo que no se recomienda su exclusión ni sustitución.

8.5 Trade-offs

La incorporación de estos controles implica ciertas compensaciones que deben considerarse:

Costo: servicios como WAF, KMS y CloudTrail añaden un gasto mensual que la arquitectura original no tenía. Es un costo justificado por la protección que aportan, pero real.

Complejidad de gestión: la autenticación (Cognito), la gestión de secretos y el control de permisos añaden componentes que deben administrarse y mantenerse, aumentando la complejidad operativa frente a la simplicidad de la versión original.

Latencia: la capa de borde (WAF, CloudFront) y la autenticación introducen pasos adicionales en cada petición, lo que puede añadir una latencia mínima. En la práctica es despreciable frente a la mejora de seguridad.

En conjunto, estas compensaciones son razonables: se intercambia simplicidad y un costo bajo por una reducción significativa del riesgo. Para una aplicación que gestiona documentos de clientes, este intercambio está plenamente justificado.

9. Conclusiones

La auditoría de seguridad realizada sobre la aplicación ArchivaCloud P-10 permitió identificar 13 hallazgos validados, distribuidos en el código de la aplicación, sus dependencias y la configuración del despliegue en la nube. El estado de seguridad general de la aplicación es deficiente: en su forma actual, no debería desplegarse en un entorno productivo con datos reales de clientes.

El riesgo más grave es la ausencia total de autenticación (VULN-001), que permite a cualquier persona acceder, modificar y eliminar todos los documentos almacenados sin necesidad de credenciales. A este se suman la posibilidad de sobrescribir archivos sin control (VULN-003) y la falta de versionado (VULN-008), que en conjunto permiten destruir información de forma irreversible y sin dejar rastro, ya que tampoco existe registro de accesos (VULN-009).

La metodología empleada demostró que ningún tipo de análisis es suficiente por sí solo. Las herramientas de análisis estático (Bandit, Semgrep) no detectaron ninguna de las vulnerabilidades de lógica o configuración; el análisis de dependencias (pip-audit) sí encontró componentes vulnerables; la revisión manual del código reveló las fallas de diseño; y las pruebas dinámicas confirmaron que esas fallas son explotables en la práctica. La combinación de estas técnicas, junto con la auditoría de la configuración cloud, fue lo que permitió obtener una visión completa del estado de seguridad de la aplicación.

Adicionalmente, la auditoría reveló una brecha importante entre la documentación de seguridad entregada por el equipo de desarrollo y la implementación real: varios controles declarados como cumplidos no estaban correctamente implementados. Esto refuerza la necesidad de auditorías externas independientes, que no se limiten a confiar en la autoevaluación del desarrollador.

Las recomendaciones de mayor impacto, en orden de prioridad, son: implementar autenticación y autorización antes de cualquier despliegue productivo; habilitar el versionado del bucket para garantizar la recuperación de los datos; migrar las credenciales a un gestor seguro de secretos aplicando el principio de mínimo privilegio; y activar el registro de accesos para asegurar la trazabilidad. La arquitectura objetivo propuesta en la Fase 6, definida mediante Infraestructura como Código, corrige estos hallazgos de forma reproducible y verificable, evitando que la seguridad dependa de configuraciones manuales.

En conjunto, los resultados confirman que ArchivaCloud requiere una intervención de seguridad significativa antes de su puesta en producción, pero también que todos los hallazgos identificados son corregibles mediante controles conocidos y de bajo costo relativo. Aplicar las recomendaciones de este informe permitiría transformar la aplicación desde su estado vulnerable actual hacia un sistema acorde a las buenas prácticas de la industria.

10. Catálogo de vulnerabilidades

Las 13 vulnerabilidades validadas se documentan a continuación con la plantilla de la evaluación. Los puntajes CVSS de las dependencias (VULN-006, 007) provienen del NVD; los de código propietario son estimaciones del auditor según el vector indicado.

VULN-001 — Ausencia de autenticación en los endpoints

ID	VULN-001
Categoría OWASP	A01:2021 Broken Access Control / API2:2023 Broken Authentication
CWE	CWE-306: Missing Authentication for Critical Function
CVE asociado	N/A (vulnerabilidad en código propietario)
Técnica MITRE	T1530 (Data from Cloud Storage); T1078.004 (Valid Accounts: Cloud)
CVSS v3.1	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N → 9.1 Crítico (estimado por el auditor)
Validación	Confirmado con curl (peticiones sin token)
Descripción	Ninguno de los tres endpoints (presigned-url, GET /api/files, DELETE /api/files/{key}) implementa autenticación ni autorización. Un cliente anónimo puede listar, generar URLs de subida y eliminar cualquier objeto.
Prueba de concepto	curl http://localhost:8000/api/files devuelve la lista completa sin credenciales (.Evidencia 12). curl -X DELETE ... elimina un objeto y responde "Archivo eliminado" (Evidencia 14).
Impacto	Técnico: lectura y borrado no autorizado de todos los documentos. Negocio: pérdida total de confidencialidad e integridad de la información gestionada. Regulatorio: incumplimiento de la Ley 19.628/21.719 sobre protección de datos.
Remediación	Implementar autenticación (p. ej. API Gateway + Cognito, o JWT/OAuth2 en el backend) y autorización a nivel de objeto. Controles ISO/IEC 27001 A.8.2 y A.8.3; alineado con CIS IAM.
Esfuerzo / Costo	Esfuerzo: alto (~16-24 h-persona: integrar autenticación en los 3 endpoints y probar). Costo AWS: bajo (Cognito dentro de capa gratuita; API Gateway ≈ 0,18 USD/mes en uso bajo).

VULN-002 — Path/Key traversal en el parámetro fileName

ID	VULN-002
Categoría OWASP	A01:2021 Broken Access Control / API1:2023 BOLA
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
CVE asociado	N/A (código propietario)
Técnica MITRE	T1565.001 — Stored Data Manipulation
CVSS v3.1	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N → 7.5 Alto (estimado)
Validación	Confirmado con curl
Descripción	El backend construye la key del objeto concatenando el nombre que envía el cliente sin sanitizar: key = f"uploads/{data.fileName}". Matiz técnico: al ser Amazon S3 (no un sistema de archivos), el vector ../../etc/passwd NO permite leer archivos del sistema; el riesgo real es escribir objetos fuera del prefijo uploads/ y sobrescribir objetos existentes dentro del bucket.

Prueba de concepto	POST con fileName="../../../escape/test.pdf" devuelve key="uploads/../../../escape/test.pdf" (Evidencia 13). El código vulnerable se observa en Evidencia 2
Impacto	Técnico: colocación/escritura de objetos fuera del prefijo previsto y base para sobrescritura (VULN-003). Negocio: corrupción de la integridad del repositorio documental.
Remediación	Sanitizar con os.path.basename(fileName) y validar contra lista blanca de caracteres (regex <code>^[w\-.]+\$</code>). Nota: os.path.abspath() NO aplica (es para filesystem). Control ISO/IEC 27001 A.8.28 (Secure coding).
Esfuerzo / Costo	Esfuerzo: bajo (~2-4 h: sanitizar el nombre de archivo). Costo AWS: nulo.

VULN-003 — Sobrescritura de objetos sin control de propiedad

ID	VULN-003
Categoría OWASP	A01:2021 Broken Access Control / API1:2023 BOLA
CWE	CWE-639: Authorization Bypass Through User-Controlled Key (rel. CWE-284)
CVE asociado	N/A (código propietario)
Técnica MITRE	T1565.001 — Stored Data Manipulation
CVSS v3.1	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N → 7.5 Alto (estimado)
Validación	Confirmado con curl + aws s3 cp
Descripción	La generación de la presigned URL no verifica si la key ya existe ni a qué usuario pertenece. Subir un objeto con el mismo nombre reemplaza el existente. Combinado con la ausencia de versioning (VULN-008), la pérdida es irreversible; combinado con VULN-001, lo ejecuta un atacante anónimo.
Prueba de concepto	Se subió un objeto "víctima" y luego otro con el mismo nombre; aws s3 cp confirma el contenido reemplazado por "ARCHIVO SOBRESCRITO POR EL ATACANTE" (Evidencia 18).
Impacto	Técnico: corrupción y pérdida de integridad de documentos. Negocio: alteración no detectable de archivos críticos.
Remediación	Namespacing de keys por usuario, verificación de propiedad antes de emitir la URL, y habilitar versioning. Control ISO/IEC 27001 A.8.3.
Esfuerzo / Costo	Esfuerzo: medio (~4-6 h: namespacing por usuario y verificación de propiedad). Costo AWS: nulo.

VULN-004 — Validación de tipo de archivo falsificable

ID	VULN-004
Categoría OWASP	A04:2021 Insecure Design
CWE	CWE-434: Unrestricted Upload of File with Dangerous Type
CVE asociado	N/A (código propietario)
Técnica MITRE	—
CVSS v3.1	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N → 5.4 Medio (estimado)
Validación	Confirmado con curl
Descripción	El backend valida data.fileType, valor declarado por el cliente, en lugar del contenido real del archivo. Es trivialmente falsificable: se acepta un .html declarándolo application/pdf.

Prueba de concepto	POST con fileName="malicioso.html" y fileType="application/pdf" genera la URL sin rechazo (Evidencia. 15). El archivo aparece almacenado en la app (Evidencia 16).
Impacto	Técnico: subida de contenido peligroso (HTML con script) disfrazado de PDF; potencial XSS si el objeto se sirve y se abre en navegador. La gravedad depende de la configuración de entrega del bucket.
Remediación	Validación del contenido real (magic bytes) del lado servidor tras la subida (evento S3 + Lambda), y verificación del Content-Type efectivo. Control ISO/IEC 27001 A.8.28.
Esfuerzo / Costo	Esfuerzo: medio (~6-8 h: validar el contenido real vía evento S3 + Lambda). Costo AWS: bajo (Lambda en capa gratuita para volumen bajo).

VULN-005 — Fuga de información en mensajes de error

ID	VULN-005
Categoría OWASP	A09:2021 Security Logging and Monitoring Failures / API8:2023
CWE	CWE-209: Generation of Error Message Containing Sensitive Information
CVE asociado	N/A (código propietario)
Técnica MITRE	—
CVSS v3.1	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N → 5.3 Medio (estimado)
Validación	Confirmado con curl
Descripción	Los tres endpoints capturan excepciones con except Exception as e y devuelven detail=str(e), reenviando al cliente el mensaje crudo de boto3 (nombre de operación, parámetros internos del SDK de AWS).
Prueba de concepto	Una key inválida en DELETE devuelve "An error occurred (400) when calling the DeleteObject operation: Bad Request" con HTTP 500 al cliente (Evidencia 17). Patrón sistémico visible en Evidencia 3, 4 y 5
Impacto	Técnico: revela el uso de boto3/S3 y operaciones internas, facilitando el reconocimiento del atacante.
Remediación	Devolver mensajes genéricos al cliente y registrar el detalle solo en logs internos. Control ISO/IEC 27001 A.8.15 (Logging).
Esfuerzo / Costo	Esfuerzo: bajo (~2 h: mensajes genéricos y registro interno). Costo AWS: nulo.

VULN-006 — Dependencia vulnerable: starlette 1.2.1

ID	VULN-006
Categoría OWASP	A06:2021 Vulnerable and Outdated Components
CWE	CWE-770 / CWE-1284 (límites de recursos no aplicados)
CVE asociado	CVE-2026-54283 (CVSS 7.5 High), CVE-2026-54282
Técnica MITRE	T1499 — Endpoint Denial of Service
CVSS v3.1	7.5 (oficial NVD, CVE-2026-54283) — confirmar vector en NVD
Validación	Confirmado con pip-audit
Descripción	starlette 1.2.1 (núcleo de FastAPI): request.form() acepta max_fields y max_part_size, pero estos límites se ignoran silenciosamente para application/x-www-form-urlencoded. Un atacante no autenticado puede enviar un cuerpo con un número arbitrario de campos o un campo gigante, evadiendo los límites configurados. Corregido en 1.3.1.

Prueba de concepto	Salida de pip-audit sobre el entorno del backend (Evidencia 6).
Impacto	Técnico: denegación de servicio por agotamiento de recursos.
Remediación	Actualizar starlette a $\geq 1.3.1$. Política de gestión de dependencias (Dependabot/Renovate). Control ISO/IEC 27001 A.8.8.
Esfuerzo / Costo	Esfuerzo: bajo (~1 h: actualizar starlette y probar). Costo AWS: nulo.

VULN-007 — Dependencia vulnerable: msgpack 1.1.2

ID	VULN-007
Categoría OWASP	A06:2021 Vulnerable and Outdated Components
CWE	CWE-248 / CWE-400 (crash por reuso tras error → DoS)
CVE asociado	GHSA-6v7p-g79w-8964 (CVSS 7.5 High)
Técnica MITRE	T1499 — Endpoint Denial of Service
CVSS v3.1	7.5 (GHSA) — confirmar vector en NVD/GitHub Advisory
Validación	Confirmado con pip-audit
Descripción	msgpack 1.1.2 (dependencia transitiva): reutilizar el Unpacker tras un error puede provocar un fallo de segmento (SEGV) y caída del proceso. Si se usa para desempaquetar entrada no confiable, es explotable como DoS. Corregido en 1.2.1.
Prueba de concepto	Salida de pip-audit (Evidencia 6).
Impacto	Técnico: caída del proceso / denegación de servicio.
Remediación	Actualizar msgpack a $\geq 1.2.1$. Control ISO/IEC 27001 A.8.8.
Esfuerzo / Costo	Esfuerzo: bajo (~1 h: actualizar msgpack). Costo AWS: nulo.

VULN-008 — Versioning del bucket deshabilitado

ID	VULN-008
Categoría OWASP	A04:2021 Insecure Design (configuración)
CWE	CWE-693: Protection Mechanism Failure
CVE asociado	N/A (configuración cloud)
Técnica MITRE	—
CVSS v3.1	AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:H → 6.5 Alto (estimado; amplifica VULN-003)
Validación	aws s3api get-bucket-versioning (salida vacía = deshabilitado)
Descripción	El versioning está deshabilitado. Sin él, la sobrescritura (VULN-003) o el borrado (VULN-001) de un objeto son irreversibles: no existe copia previa recuperable.
Prueba de concepto	Evidencia 7 (auditoría del bucket, sección Versioning).
Impacto	Técnico: pérdida irreversible de datos. Negocio: imposibilidad de recuperación ante manipulación o error.
Remediación	Habilitar versioning y MFA Delete. Definir RPO. CIS AWS (almacenamiento S3). NIST CSF: Recover.
Esfuerzo / Costo	Esfuerzo: bajo (~1 h: habilitar versionado). Costo AWS: bajo (almacenamiento de versiones adicionales, según volumen).

VULN-009 — Sin server access logging en el bucket

ID	VULN-009
Categoría OWASP	A09:2021 Security Logging and Monitoring Failures
CWE	CWE-778: Insufficient Logging
CVE asociado	N/A (configuración cloud)
Técnica MITRE	T1562.008 (gap defensivo)
CVSS v3.1	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N → 3.1 Medio-Bajo (estimado)
Validación	aws s3api get-bucket-logging (salida vacía = sin logging)
Descripción	El bucket no tiene server access logging ni CloudTrail data events. No existe trazabilidad de quién accede o modifica objetos, impidiendo la investigación de incidentes.
Prueba de concepto	Evidencia 7 (sección Server access logging).
Impacto	Técnico: ausencia de capacidad forense y de detección.
Remediación	Habilitar S3 server access logging y CloudTrail data events; centralizar en CloudWatch/SIEM. CIS AWS (logging). NIST CSF: Detect.
Esfuerzo / Costo	Esfuerzo: bajo (~2-3 h: habilitar logging + CloudTrail data events). Costo AWS: ≈ 1,5 USD/mes.

VULN-010 — Cifrado SSE-S3 en lugar de SSE-KMS/CMK

ID	VULN-010
Categoría OWASP	A02:2021 Cryptographic Failures
CWE	CWE-311: Missing Encryption of Sensitive Data (gestión de clave)
CVE asociado	N/A (configuración cloud)
Técnica MITRE	—
CVSS v3.1	AV:L/AC:H/PR:H/UI:N/S:U/C:H/I:N/A:N → 4.2 Medio (estimado)
Validación	aws s3api get-bucket-encryption (SSEAlgorithm: AES256, BucketKeyEnabled: false)
Descripción	El bucket cifra con SSE-S3 (clave gestionada por AWS). Para datos sensibles se recomienda SSE-KMS con clave gestionada por el cliente (CMK), que permite control de rotación, políticas de uso y trazabilidad del uso de la clave en CloudTrail.
Prueba de concepto	Evidencia 7 (sección Cifrado en reposo).
Impacto	Técnico: menor control criptográfico y de cumplimiento sobre la clave.
Remediación	Migrar a SSE-KMS con CMK propia y habilitar Bucket Key. CIS AWS 2.1 (cifrado S3).
Esfuerzo / Costo	Esfuerzo: medio (~2-4 h: crear la CMK y migrar el cifrado). Costo AWS: ≈ 1 USD/mes (CMK) + uso de API.

VULN-011 — Sin bucket policy que fuerce TLS

ID	VULN-011
Categoría OWASP	A05:2021 Security Misconfiguration
CWE	CWE-319: Cleartext Transmission of Sensitive Information

CVE asociado	N/A (configuración cloud)
Técnica MITRE	—
CVSS v3.1	AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N → 5.9 Medio (estimado)
Validación	aws s3api get-bucket-policy (NoSuchBucketPolicy)
Descripción	El bucket no tiene política que deniegue las conexiones no cifradas (condición aws:SecureTransport=false). En consecuencia, no se garantiza el uso obligatorio de TLS y no existe restricción adicional a nivel de bucket.
Prueba de concepto	Evidencia 7 (sección Bucket policy).
Impacto	Técnico: posible transmisión en claro y exposición a man-in-the-middle.
Remediación	Añadir bucket policy con Deny cuando aws:SecureTransport sea false; exigir TLS 1.2+. CIS AWS 2.1 (deny HTTP).
Esfuerzo / Costo	Esfuerzo: bajo (~1 h: aplicar la bucket policy). Costo AWS: nulo.

VULN-012 — Credenciales en .env / IAM sin mínimo privilegio

ID	VULN-012
Categoría OWASP	A05:2021 Security Misconfiguration / A07:2021 Identification and Authentication Failures
CWE	CWE-522: Insufficiently Protected Credentials
CVE asociado	N/A (configuración cloud)
Técnica MITRE	T1552.001 (Credentials In Files); T1078.004
CVSS v3.1	AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H → 7.0 Alto (estimado)
Validación	aws sts get-caller-identity (rol voclabs); credenciales en backend/.env
Descripción	La aplicación carga credenciales AWS desde un archivo .env en texto plano (anti-patrón). En el laboratorio son temporales del rol voclabs, que además es amplio y no respeta el mínimo privilegio; en producción serían claves permanentes en archivo. La política mínima documentada (4 permisos S3) no se aplica realmente.
Prueba de concepto	Evidencia 8 (identidad IAM en uso).
Impacto	Técnico: la filtración del .env compromete la cuenta; el exceso de privilegios amplía el radio de impacto.
Remediación	Usar instance profiles (EC2) o IRSA (EKS), AWS Secrets Manager, política de mínimo privilegio (solo PutObject/GetObject/DeleteObject/ListBucket sobre el ARN correcto) y rotación de credenciales. CIS AWS (IAM).
Esfuerzo / Costo	Esfuerzo: medio (~6-8 h: Secrets Manager + rol de mínimo privilegio + refactor). Costo AWS: ≈ 0,40 USD/mes por secreto.

VULN-013 — Validación de seguridad solo del lado cliente

ID	VULN-013
Categoría OWASP	A04:2021 Insecure Design
CWE	CWE-602: Client-Side Enforcement of Server-Side Security
CVE asociado	N/A (código propietario)
Técnica MITRE	—
CVSS v3.1	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N → 5.3 Medio (estimado)

Validación	Confirmado con curl (el backend acepta lo que el frontend rechaza)
Descripción	El frontend valida el tipo (allowedTypes) y el tamaño (MAX_SIZE = 22 MB), pero es una validación cosmética: se evita con una petición directa al backend, y además la subida real va directo a S3 mediante la presigned URL. El backend no replica una validación efectiva (ver VULN-004).
Prueba de concepto	Las pruebas con curl de VULN-002 y VULN-004 evaden el frontend (Evidencia 13, 15).
Impacto	Técnico: falsa sensación de seguridad; los controles del cliente son trivialmente evitables.
Remediación	Trasladar la validación autoritativa al servidor; el cliente solo para experiencia de usuario. Control ISO/IEC 27001 A.8.28.
Esfuerzo / Costo	Esfuerzo: bajo (~4 h: trasladar la validación al servidor). Costo AWS: nulo.

11. Bibliografía (APA 7)

- Amazon Web Services. (2023). *AWS Well-Architected Framework: Security pillar*. <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/>
- Amazon Web Services. (s.f.). *Amazon S3 user guide*. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
- Amazon Web Services. (s.f.). *AWS Identity and Access Management user guide*. <https://docs.aws.amazon.com/IAM/latest/UserGuide/>
- Center for Internet Security. (2024). *CIS Amazon Web Services Foundations Benchmark* (Versión 3.0.0). CIS.
- Donweb Cloud. (2025, abril 11). *CURL: La navaja suiza que necesitas si eres programador* [Video]. YouTube. <https://www.youtube.com/watch?v=n3NtrQYrjDw>
- Fazt Code. (2019, julio 6). *cURL (Client URL): Protocolos de Internet desde consola* [Video]. YouTube. <https://www.youtube.com/watch?v=9u2qDQc6KWg>
- FIRST. (2019). *Common Vulnerability Scoring System version 3.1: Specification document*. <https://www.first.org/cvss/>
- HackerSploit. (2021, mayo 6). *Dumping S3 buckets: Exploiting S3 bucket misconfigurations* [Video]. YouTube. <https://www.youtube.com/watch?v=ITSZ8743MUK>
- Igp Training. (2026, abril 18). *6 pasos para realizar una auditoría de ciberseguridad basada en NIST 2.0* [Video]. YouTube. <https://www.youtube.com/watch?v=NxayR7c2cZE>
- International Organization for Standardization. (2022). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (ISO/IEC 27001:2022). ISO.
- MITRE Corporation. (2024). *MITRE ATT&CK for Cloud*. <https://attack.mitre.org/matrices/enterprise/cloud/>
- National Institute of Standards and Technology. (2024). *The NIST Cybersecurity Framework (CSF) 2.0* (NIST CSWP 29). <https://doi.org/10.6028/NIST.CSWP.29>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- OWASP Foundation. (2022). *OWASP Application Security Verification Standard* (Versión 4.0.3). <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Foundation. (2023). *OWASP API Security Top 10 2023*. <https://owasp.org/API-Security/>
- PyCQA. (s.f.). *Bandit: Security oriented static analyser for Python code*. <https://bandit.readthedocs.io/>
- Python Packaging Authority. (s.f.). *pip-audit: Audit Python environments for known vulnerabilities*. <https://pypi.org/project/pip-audit/>
- Semgrep, Inc. (s.f.). *Semgrep documentation*. <https://semgrep.dev/docs/>
- Spartan-Cybersecurity. (2022, septiembre 10). *Aprende a detectar vulnerabilidades en buckets S3 - Parte 1* [Video]. YouTube. <https://www.youtube.com/watch?v=kp8TEk8hZGc>

Anexos

Anexo A — Evidencias

Evidencia 1 Bandit: sin hallazgos (Fase 1)

```
(venv) C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud\backend>type ..\bandit_report.txt
Run started:2026-06-26 14:30:08.562891+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 111
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):

(venv) C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud\backend>semgrep --config=auto app
```

Evidencia 2 Código: `key=f"uploads/{fileName}"` (VULN-002)

```
key = f"uploads/{data.fileName}"

try:
    presigned_url = s3_client.generate_presigned_url(
        "put_object",
        Params={
            "Bucket": os.getenv("S3_BUCKET_NAME"),
            "Key": key,
            "ContentType": data.fileType
        },
        ExpiresIn=300
    )

    return {
        "presignedUrl": presigned_url,
        "key": key
    }

except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=str(e)
    )
```

Evidencia 3 Código: endpoint DELETE (VULN-001/005)

```
#Endpoint DELETE:
@app.delete("/api/files/{key:path}")
def delete_file(key: str):

    try:

        s3_client.delete_object(
            Bucket=os.getenv("S3_BUCKET_NAME"),
            Key=key
        )

        return {
            "message": "Archivo eliminado"
        }

    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=str(e)
        )
```

Evidencia 4 presigned-url + str(e) (VULN-002/004/005)

```
# Endpoint: Presigned URL
@app.post("/api/upload/presigned-url")
def generate_presigned_url(data: UploadRequest):

    allowed_types = [
        "application/pdf",
        "text/csv",
        "application/vnd.ms-excel"
    ]

    if data.fileType not in allowed_types:
        raise HTTPException(
            status_code=400,
            detail="Tipo de archivo no permitido"
        )

    MAX_SIZE = 22 * 1024 * 1024

    if data.fileSize > MAX_SIZE:
        raise HTTPException(
            status_code=400,
            detail="El archivo supera el límite de 22 MB"
        )

    key = f"uploads/{data.fileName}"

    try:
        presigned_url = s3_client.generate_presigned_url(
            "put_object",
            Params={
                "Bucket": os.getenv("S3_BUCKET_NAME"),
                "Key": key,
                "ContentType": data.fileType
            },
            ExpiresIn=300
        )

        return {
            "presignedUrl": presigned_url,
            "key": key
        }

    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=str(e)
        )
```

Evidencia 5 Código: list_files (VULN-001/005)

```
# Endpoint Get files:
@app.get("/api/files")
def list_files():
    try:
        response = s3_client.list_objects_v2(
            Bucket=os.getenv("S3_BUCKET_NAME"),
            Prefix="uploads/"
        )

        files = []

        for obj in response.get("Contents", []):
            if obj["Key"] == "uploads/": #para que no cuente la carpeta uploads xD
                continue

            files.append({
                "key": obj["Key"],
                "size": obj["Size"],
                "lastModified": obj["LastModified"]
            })

        return files

    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=str(e)
        )
```

Evidencia 6 pip-audit: 8 vuln. en 3 paquetes (VULN-006/007)

1	Found 8 known vulnerabilities in 3 packages			
2	Name	Version	ID	Fix Versions
3	-----	-----	-----	-----
4	msgpack	1.1.2	GHSA-6v7p-g79w-8964	1.2.1
5	pip	25.1.1	PYSEC-2026-196	26.1.2
6	pip	25.1.1	CVE-2025-8869	25.3
7	pip	25.1.1	CVE-2026-1703	26.0
8	pip	25.1.1	CVE-2026-3219	26.1
9	pip	25.1.1	CVE-2026-6357	26.1
10	starlette	1.2.1	CVE-2026-54283	1.3.1
11	starlette	1.2.1	CVE-2026-54282	1.3.0
12				

Evidencia 7 Auditoría bucket: cifrado, versioning, BPA, logging, lifecycle, policy (Fase 4)

```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>type evidencias\fase4_cloud.txt
=====
AUDITORIA CLOUD - FASE 4 - Bucket: archivacloud-p10-jz
=====

[1] CIFRADO EN REPOSO
COMANDO: aws s3api get-bucket-encryption --bucket archivacloud-p10-jz
--- SALIDA:
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "AES256"
        },
        "BucketKeyEnabled": false,
        "BlockedEncryptionTypes": {
          "EncryptionType": [
            "SSE-C"
          ]
        }
      }
    ]
  }
}

[2] VERSIONING
COMANDO: aws s3api get-bucket-versioning --bucket archivacloud-p10-jz
--- SALIDA (vacía = DESHABILITADO):

[3] BLOCK PUBLIC ACCESS
COMANDO: aws s3api get-public-access-block --bucket archivacloud-p10-jz
--- SALIDA:
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}

[4] SERVER ACCESS LOGGING
COMANDO: aws s3api get-bucket-logging --bucket archivacloud-p10-jz
--- SALIDA (vacía = SIN LOGGING):

[5] LIFECYCLE POLICY
COMANDO: aws s3api get-bucket-lifecycle-configuration --bucket archivacloud-p10-jz
--- SALIDA:

aws: [ERROR]: An error occurred (NoSuchLifecycleConfiguration) when calling the GetBucketLifecycleConfiguration operation: The lifecycle configuration does not exist

[6] BUCKET POLICY
COMANDO: aws s3api get-bucket-policy --bucket archivacloud-p10-jz
--- SALIDA:

aws: [ERROR]: An error occurred (NoSuchBucketPolicy) when calling the GetBucketPolicy operation: The bucket policy does not exist
```

Evidencia 8 Auditoría: IAM (voclabs) y CORS del bucket (VULN-012)

```
[7] IAM - IDENTIDAD EN USO
COMANDO: aws sts get-caller-identity
--- SALIDA:
{
  "UserId": "AROA2UC3BZIWLQYMR5BQ:user4921928=joaquin.zambrano02@inacapmail.cl",
  "Account": "730335398444",
  "Arn": "arn:aws:sts::730335398444:assumed-role/voclabs/user4921928=joaquin.zambrano02@inacapmail.cl"
}

[8] CORS DEL BUCKET
COMANDO: aws s3api get-bucket-cors --bucket archivacloud-p10-jz
--- SALIDA:
{
  "CORSRules": [
    {
      "AllowedHeaders": [
        "*"
      ],
      "AllowedMethods": [
        "GET",
        "POST",
        "PUT",
        "DELETE"
      ],
      "AllowedOrigins": [
        "http://localhost:5173"
      ]
    }
  ]
}

C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

Evidencia 9 npm audit: 0 vulnerabilidades (Fase 1)

```
(venv) C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud\backend>cd ../frontend
(venv) C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud\frontend>npm audit
found 0 vulnerabilities
(venv) C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud\frontend>
```


Evidencia 10 Semgrep: 0 hallazgos (Fase 1)

Scan Summary

```
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 290
• Targets scanned: 3
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 290 rules on 3 files: 0 findings.
💎 Missed out on 1860 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)
```

Evidencia 11 DELETE sobre inexistente → "Archivo eliminado" (observación)

```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl -X DELETE "http://localhost:8000/api/files/uploads/no-existe-xyz.pdf"
{"message":"Archivo eliminado"}
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

Evidencia 12 GET /api/files sin token (VULN-001)

```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl http://localhost:8000/api/files
[{"key":"uploads/contrato.pdf","size":38,"lastModified":"2026-06-26T15:00:56+00:00"},
{"key":"uploads/sample.pdf","size":18810,"lastModified":"2026-06-26T15:14:26+00:00"}]
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl -X POST "http://localhost:8000/api/upload/presigned-url" -H "Content-Type: application/json" -d "{\"fileName\":\"../../escape/test.pdf\",\"fileType\":\"application/pdf\",\"fileSize\":1000}"
{"presignedUrl":"https://archivacloud-p10-jz.s3.amazonaws.com/uploads/../../escape/test.pdf?AWSAccessKeyId=AKIAI44QH8DHBVS7JL5N6&Signature=whLL%...&content-type=application%2Fpdf&x-amz-security-token:..."}
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl -X DELETE "http://localhost:8000/api/files/uploads/sample.pdf"
{"message": "Archivo eliminado"}
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

Evidencia 15 html aceptado como PDF (VULN-004)

```
Símbolo del sistema
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl -X POST "http://localhost:
8000/api/upload/presigned-url" -H "Content-Type: application/json" -d "{\"fileName\":\"ma
licioso.html\",\"fileType\":\"application/pdf\",\"fileSize\":\"1000\"}"
{"presignedUrl":"https://archivacloud-p10-jz.s3.amazonaws.com/uploads/malicioso.html?AWSA
ccessKeyId=&Signature=&content-type=app
lication%2Fpdf&x-amz-security-token="
{"res=1782488322","key":"uploads/malicioso.html"}
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```

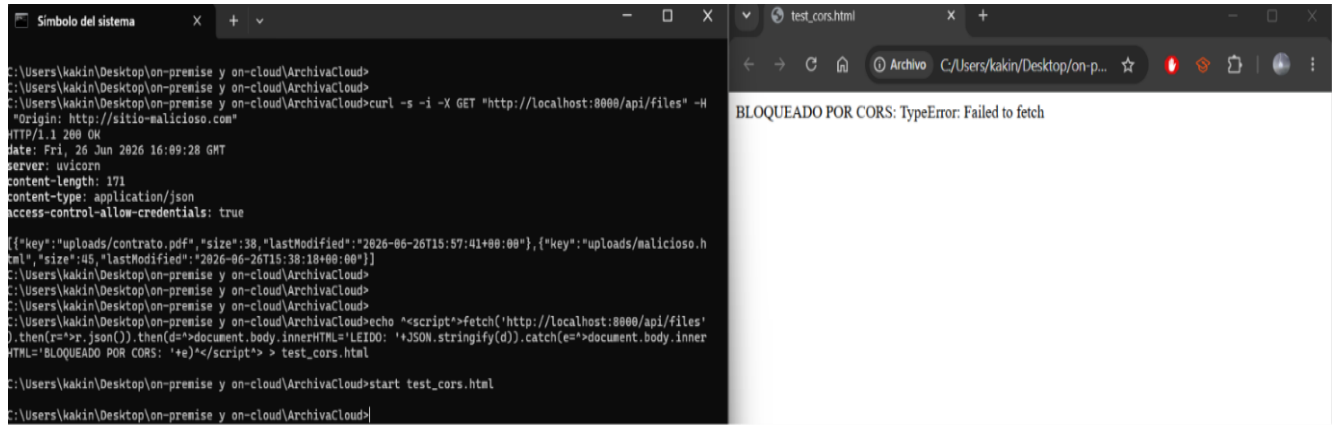


```
C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>curl -s -X DELETE "http://localhost:8000/api/files/uploads/%00invalid" -w "\nHTTP: %{http_code}\n" > evidencias\prueba5_fuga_info.txt

C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>type evidencias\prueba5_fuga_info.txt
{"detail":"An error occurred (400) when calling the DeleteObject operation: Bad Request"}
HTTP: 500

C:\Users\kakin\Desktop\on-premise y on-cloud\ArchivaCloud>
```


Evidencia 19 CORS: origen externo bloqueado (control correcto)



Anexo B — Matriz STRIDE completa

Se presenta la matriz Stride completa además de enviarla como documento adicional en formato Excel

Matriz STRIDE — ArchivaCloud (S3 File Upload)

Modelado de amenazas sobre el DFD nivel 1. Cada amenaza se cruza con el hallazgo validado correspondiente.

Elemento del DFD	Categoría STRIDE	Amenaza	Hallazgo asociado	Severidad
Entidad: Navegador	S — Spoofing	Sin autenticación: cualquiera se hace pasar por un usuario válido.	VULN-001	Crítico
Entidad: Navegador	T — Tampering	El cliente manipula fileName, fileType y fileSize enviados al backend.	VULN-002, 004, 013	Alto
Entidad: Navegador	R — Repudiation	Acciones sin identidad ni registro: no son atribuibles a un usuario.	VULN-001, 009	Medio
Proceso: Backend FastAPI	E — Elevation of privilege	Endpoints críticos accesibles sin ningún control de acceso.	VULN-001	Crítico
Proceso: Backend FastAPI	I — Information disclosure	detail=str(e) expone errores internos de boto3 al cliente.	VULN-005	Medio
Proceso: Backend FastAPI	D — Denial of Service	Sin rate limiting; la dependencia starlette amplifica el riesgo de DoS.	VULN-006	Medio
Almacén: Bucket S3	T — Tampering	Traversar y sobrescritura de objetos; sin versioning la pérdida es irreversible.	VULN-002, 003, 008	Alto
Almacén: Bucket S3	I — Information disclosure	Sin logging de accesos; cifrado sin clave propia (CMK).	VULN-009, 010	Medio
Almacén: .env (credenciales)	I — Information disclosure	Credenciales AWS almacenadas en texto plano en el servidor.	VULN-012	Alto
Almacén: .env (credenciales)	E — Elevation of privilege	La filtración del .env conduce al compromiso de la cuenta AWS.	VULN-012	Alto
Flujo: Navegador → S3 (directo)	T — Tampering	Subida sin validación del servidor: se evita el backend (presigned URL).	VULN-002, 004	Alto
Flujo: Backend ↔ S3	I — Information disclosure	Sin política que fuerce TLS en el transporte hacia S3.	VULN-011	Medio

Figura 3 Matriz STRIDE Completa

Anexo C — Código IaC

Por motivos de extensión, este anexo presenta los fragmentos más representativos del código Terraform. El archivo completo (main.tf) se entrega como parte del repositorio Git (entregable 4), conforme a lo solicitado.

Anexo D — Declaración de uso de IA generativa

Declaración de uso de inteligencia artificial generativa

En el desarrollo de este trabajo se utilizó la herramienta de IA generativa Claude (Anthropic) como apoyo en las siguientes tareas:

- Organización y estructura del informe (índice, títulos y formato).
- Corrección y sugerencias enfocadas en redacción, ortografía, signos y gramática
- Consultor Especifico para el montaje del entorno de laboratorio y la ejecución de las pruebas.

- Consultor de estructura y contenido para la Redacción de las fichas técnicas de vulnerabilidad y mapeo a marcos (OWASP, CWE, MITRE, CIS).
- Apoyo en la Generación del diagrama de flujo de datos (DFD).
- Redacción de Resumen ejecutivo en base al contenido desarrollado o contexto de pruebas

Validación de los resultados: todas las pruebas dinámicas y de configuración fueron ejecutadas realmente por el estudiante sobre su propio entorno, y sus salidas (capturas y archivos de evidencia) confirman cada hallazgo. La redacción final y la revisión del contenido son responsabilidad del estudiante.

Firma: Joaquin Eduardo Zambrano Baeza Fecha: 30/ 06 / 2026